Foldback

Handoff for the Mac mini host and the Claude helping them

Written 29 October 2026 by Nishit Prajapati's team. PayPal AI Hackathon entry. Submission deadline Thursday 12 November 2026, 12:00 PT (we aim to submit on 11 November). Judging runs 1 to 15 December 2026.

STEP 0, before anything else: get the latest repository. This document was written from the code as it is on the `main` branch today, and the project changes every day. If you have never cloned it:

```
git clone https://github.com/AnujSaha0111/PayPal-Hackathon.git
```

Then, every time you start work:

```
cd PayPal-Hackathon
git checkout main
git pull
```

Then read `docs/spec/README.md` and `docs/handoff/` for anything newer than this PDF.

How to use this document

- **You are the owner of the Mac mini.** Your job is to turn it into a small, reliable server that hosts Foldback's database and backend. Section 5 is the step-by-step setup. Sections 1 to 4 explain what the project is, so the setup makes sense.
- **If you use Claude** (Claude Code or the Claude app), give it this whole PDF. Section 8 contains the rules it must follow.
- Send Nishit the facts listed in section 9 as you go. Never send passwords, tokens or keys in chat. Section 6 says how secrets are handled.

1. What Foldback is

Foldback is a group payment where nobody pays unless the whole group makes it. Example: four friends want a cabin for 320 dollars, 80 dollars each, deadline Friday 6 pm. Each friend opens a link and approves a **PayPal authorization hold** for their share. A hold reserves money but does not take it. If all four holds exist by the deadline, Foldback **captures** every hold. If not, Foldback **voids** every hold and nobody is charged.

Two AI features sit on top, and neither one moves money:

- **The rule reader.** The organizer pastes the group chat. A language model turns it into a structured plan (price, people, deadline, optional grace period for one person) and quotes the line it used for each field. Code checks the math. Nothing is active until the organizer confirms.
- **The organizer assistant.** It can simulate what would happen, draft reminders, and propose moving the demo clock. It only returns proposals. Code makes every money decision (a pure function, `domain/decide`).
- The organizer sees a **Bryntum Scheduler board** (one row per person across the deadline) that updates live through Socket.IO when a friend pays on their phone.
- All payments use the **PayPal sandbox only**. No real money ever moves. The server refuses to start if `PAYPAL_ENV` is anything but `sandbox`.

Goals, equal weight: win the hackathon (judged on the PayPal and Bryntum prizes), and build it like a production system: tested, secured, deployed and observable.

Team: Nishit Prajapati (project lead), Anuj Shah, Manveer Anand, Ravi Kumavat. The Mac mini belongs to one of them (name to be filled in by Nishit).

How a payment flows

Step	What happens	Where
1	Organizer pastes the chat, reviews the extracted plan, confirms it	Frontend and backend, LLM call
2	Each friend opens /pay/{shareId} on a phone and taps Hold my share with PayPal	Frontend, backend, PayPal sandbox

Step	What happens	Where
3	PayPal returns the friend to the app; the backend authorizes the order and stores the authorization	Backend, Postgres
4	The board flips that row to Held within a second	Socket.IO event, board refetch
5	At the deadline (or when the last friend holds), the resolve step runs under a database lock: all held and the headcount met means capture all; otherwise void all	Backend, Postgres, PayPal
6	If PayPal gives an unclear answer, the share goes to an unknown status, the money stays reserved, and a sync step checks it. Nothing is guessed	Backend

2. Technology and where the code lives

Layer	Choice	Notes
Backend	Node 24, TypeScript (strict), Express 5, Socket.IO, zod 4	Layers: routes, controllers, services, repositories, db. SQL only in repositories
Database	PostgreSQL 16, plain pg driver, numbered SQL migrations with checksums	Rules live in the schema (constraints and triggers). Two roles: an owner for migrations, foldback_app (DML only) for the running app
Frontend	Next.js 16 App Router, MUI 7, Redux Toolkit and RTK Query, Bryntum Scheduler (trial)	Built with next dev and next build using the --webpack flag (Turbopack crashes on Bryntum)
AI	Free tiers only: Gemini Flash models, then Groq, then OpenRouter free models, called with plain fetch	Never add a paid provider, a card-backed key or an AI SDK
Package manager	Yarn 4.9.2 through Corepack	enableScripts is false; install scripts are allow-listed one by one
Tests	Jest 30 with ts-jest; Postgres integration tests	Needs a throwaway Postgres on port 5433

Repository map

Path	What is there
docs/spec/	The specification. README.md (status table), quality.md (permanent rules), architecture.md, api.md, data-model.md, paypal.md, ai.md, scheduler.md, ui.md, deploy.md, test.md, and one file per phase in phases/
docs/design/	The approved HTML mockups and design tokens (sharp corners, IBM Plex, sky blue)
docs/handoff/	This document
backend/	Express API, migrations (migrations/0001_init.sql), scripts, tests
deploy/	compose.dev.yml (local Postgres for development and tests). Production files arrive in sub-phases 3e and 3f
spike/	Throwaway phase-0 experiments (PayPal, LLM, Bryntum). Do not build on them

3. Where we are in development (29 October 2026)

Phase	State	What it contains
0 Spike	Done	Proved PayPal authorize then capture or void in the sandbox, tested the free LLM chain, rendered a Bryntum board

Phase	State	What it contains
1 Database design	Done	Full schema with constraints and triggers, migrator, 31 schema tests, independently reviewed
2 UI design	Done	Five clickable HTML screens in docs/design/: landing (with scroll animations), sign in, plans, board, pay. Light and dark theme, logo, tokens
3 Backend foundation	In progress	3a done and merged. 3b to 3f below
4 Rule reader	Next after 3	LLM extraction, a bake-off to pick the best free model order
5 Frontend and board	After 3	The real Next.js app on Vercel, Bryntum board, Redux store
6 to 9	Later	Pay link and hold, resolve (keep or release), assistant, ship (README, demo video, Devpost submission before 12 November 12:00 PT)

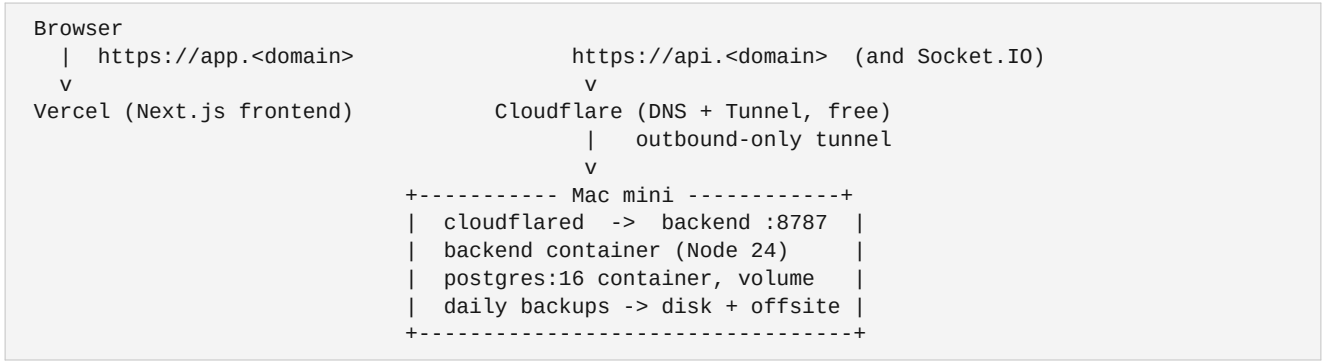
Phase 3 in detail (backend foundation and first deploy)

Sub-phase	Scope	State
3a	Environment validation (fails fast), logger that hides secrets, request ids, error handling, health endpoints, database pool	Merged. 52 tests pass. Endpoints: /health/live, /health/ready, /api/health
3b	Database transaction and advisory-lock helpers, users repository	Next
3c	Signup, login, guest accounts, cookie session, Origin check, rate limits, sample data, reset	After 3b
3d	Contracts list and read, Socket.IO handshake and rooms	After 3c
3e	Quality tooling (knip, jscpd, husky, gitleaks), backend Dockerfile, GitHub Actions CI, images to ghcr.io	After 3d
3f	Deploy to the Mac mini: Compose, Postgres, tunnel, backups, smoke test	After 3e. **This is where the Mac mini is needed.** Part A of section 5 can start now

The backend is not yet deployable. There is no production Dockerfile or Compose file until 3e and 3f. That is why section 5 is split: Part A prepares the machine now, Part B is the deployment, which is completed with exact commands from the repository when 3f lands.

4. Target architecture

The plan changed on 29 October. We dropped the earlier Azure plan. The final layout:



Item	Decision
Frontend	Vercel, built from GitHub. Served at app.<domain>
Backend and database	Docker Compose on the Mac mini. Postgres is never exposed to the network: it binds to 127.0.0.1 only

Item	Decision
Public access	A Cloudflare Tunnel. The Mac makes an outbound connection to Cloudflare, so **no router port-forwarding and no open inbound ports**. Cloudflare provides HTTPS. Served at api.<domain>
Domain	A free student domain (GitHub Student Developer Pack, Namecheap), with DNS on Cloudflare. Nishit owns the Cloudflare account and the domain
Cookies	The session cookie is set for the parent domain, so app. and api. are the same site. CORS allows only https://app.<domain>
Deploys	Pull model: the Mac checks for a new image and restarts the stack. We do not run a GitHub self-hosted runner, because the repository is public (see section 7)
Scale-out	Dropped. One machine is enough for a demo

5. Setting up the Mac mini

Do Part A now. It needs no project code beyond a clone. Part B happens when sub-phase 3f lands; Nishit or his Claude will tell you, and the exact files and commands will be in the repository under `deploy/`. Run commands in the Terminal app. Lines starting with `sudo` ask for your Mac password. Run commands one at a time and check the result before the next.

Part A. Prepare the machine (do now)

A1. Collect facts and send them to Nishit

```
sw_vers
uname -m
sysctl -n machdep.cpu.brand_string
sysctl -n hw.memsize
df -h /
fdesetup status
```

Expected: `uname -m` prints `arm64` on Apple Silicon (this decides which Docker images CI builds). `hw.memsize` is bytes of RAM. You want at least 8 GB RAM and 30 GB free disk. Send Nishit the outputs, but nothing else about the machine.

A2. Updates and a dedicated account

- Install pending macOS updates now: System Settings, General, Software Update. Reboot, then check again until it is up to date.
- Afterwards, turn off automatic installation of macOS updates (the Info button next to Automatic updates). A surprise reboot in the middle of judging would take the demo offline. Apply updates by hand at a quiet time, and tell Nishit when you do.
- Optional but recommended: create a separate standard user named `foldback` (System Settings, Users and Groups) that runs the stack, so your personal files are not mixed with the server. If you prefer to use your own account, that works too; tell Nishit which.

A3. Make it stay awake and recover from power cuts

```
sudo pmset -a sleep 0 disksleep 0 displaysleep 15 womp 1 autorestart 1
pmset -g
```

`sleep 0` means the Mac never sleeps. `autorestart 1` makes it start again after a power failure. `womp 1` allows wake over the network. In `pmset -g` check that `sleep` shows 0 and `autorestart` shows 1. Use a wired Ethernet connection if possible. A UPS (battery backup) is nice to have.

A4. Auto-login and FileVault (a decision to make with Nishit)

When the Mac restarts, containers only start after a user is logged in. **FileVault** (disk encryption) forces a password at the login screen after every reboot, which blocks automatic login. Two choices:

Choice	Result	Risk
FileVault off, Automatic login on (System Settings, Users and Groups)	After a power cut or reboot the stack comes back by itself	The disk is not encrypted. Acceptable only because the data is PayPal sandbox data and the Mac is physically safe
FileVault on	Someone must type the password after each reboot, then the stack starts	Safest for data, but an unattended reboot means downtime

Recommended for this project: FileVault off and Automatic login on, **if** the Mac is in a private place. Tell Nishit what you chose. Check the current state with `fdesetup status`.

A5. Developer tools, Git and Homebrew

```
xcode-select --install
git --version
```

Install Homebrew from its official page, **brew.sh**, following the instructions there. Read what the installer does before you run it, and do not paste commands from anywhere else. Then:

```
brew --version
brew install cloudflared
cloudflared --version
```

A6. Docker

Install **OrbStack** (recommended: light on memory, starts at login, free for personal use) or Docker Desktop. Either provides `docker` and `docker compose`.

```
brew install --cask orbstack
```

- Open OrbStack once and finish its first-run steps. In its settings turn on **Start at login**. Set a memory limit of about 4 to 6 GB.
- If you choose Docker Desktop instead: `brew install --cask docker`, open it, and tick **Start Docker Desktop when you sign in**.

```
docker --version
docker compose version
docker run --rm postgres:16 uname -m
```

The last command downloads Postgres and prints `aarch64` on Apple Silicon, which proves the machine runs the right image type.

A7. Get the repository and test the database

```
cd ~
git clone https://github.com/AnujSaha0111/PayPal-Hackathon.git
cd PayPal-Hackathon
git checkout main
git pull
docker compose -f deploy/compose.dev.yml up -d
docker compose -f deploy/compose.dev.yml ps
```

Expected: two services, `postgres` and `postgres-test`, both **healthy** after about 15 seconds. This is a development database only. It proves Docker and Postgres work on this Mac. Stop it later with:

```
docker compose -f deploy/compose.dev.yml down
```

A8. Optional: run the backend tests on the Mac

This proves the machine can run the project and shows its speed. Install Node 24 (`brew install node@24` or the installer from nodejs.org), then:

```
corepack enable
cd ~/PayPal-Hackathon/backend
corepack yarn install
docker compose -f ../deploy/compose.dev.yml up -d
corepack yarn test
```

Expected: `Tests: 52 passed` (the count grows as the project grows). Send Nishit how long it took.

A9. Remote access for Nishit and the team

- System Settings, General, Sharing, turn on **Remote Login** (SSH). Allow only your account.
- Ask Nishit for his SSH **public** key (never a private key) and add it to `~/.ssh/authorized_keys`. Create the folder with `mkdir -p ~/.ssh && chmod 700 ~/.ssh` and the file with `chmod 600 ~/.ssh/authorized_keys`.
- After key login works, turn off password logins for SSH. Put `PasswordAuthentication no` in a new file `/etc/ssh/sshd_config.d/100-keys-only.conf` (use `sudo`), then confirm you can still log in with the

key from a second terminal **before** closing the first.

- SSH is for the LAN or a private network only. Do not forward port 22 on your router. If Nishit needs access from outside your home, use Tailscale (free): `brew install --cask tailscale`, sign in, and share the machine with him.
- Turn on the macOS firewall (System Settings, Network, Firewall). The tunnel is outbound-only, so nothing needs to be opened inbound.

A10. A place for backups

- Plug in an external disk (at least 64 GB) and name it `FoldbackBackup`. Backups will be written to `/Volumes/FoldbackBackup/foldback/`.
- Agree with Nishit on one off-site copy (for example an encrypted copy to a cloud drive). A single machine is a single point of loss: if the Mac fails, the external disk and the off-site copy are the only way back.

Part B. Deploy Foldback (when sub-phase 3f lands)

The steps below are the plan. The exact commands and files (`deploy/compose.yml`, `deploy/scripts/*.sh`, the `.env` template) ship in the repository, so run `git pull` first and follow the files, not memory.

Step	What you do	Result
B1	Create `~/foldback`. Copy `deploy/` into it	A working folder that is not the git clone
B2	Create `~/foldback/.env` from the template and `chmod 600` it. Generate secrets with `openssl rand -base64 48`. Nishit gives the PayPal sandbox keys and the AI keys through a password manager, never chat	A private env file, never committed
B3	`docker compose up -d` starts Postgres, runs the migration job, then starts the backend	Containers healthy, migrations applied
B4	Create the app database role `foldback_app` with a random password (the guide in the repo has the exact SQL)	The app connects with least privilege
B5	Nishit creates the tunnel in the Cloudflare dashboard (Zero Trust, Networks, Tunnels) and sends you the install command privately. You run it once with sudo. Public hostname api.<domain> points to http://localhost:8787	Tunnel shows Healthy in Cloudflare
B6	`deploy/scripts/smoke.sh https://api.<domain>`	Health and a guest sign-in succeed
B7	Install the launchd job that checks for new backend images every few minutes and restarts the stack	Deploys happen without opening inbound access
B8	Install the launchd job for a daily `pg_dump` to the backup disk, keep 14 days, and the off-site copy	Backups exist, and a restore has been tried once
B9	Reboot the Mac and confirm everything returns by itself	Auto-login, Docker, containers and tunnel all back

Environment variables the backend needs (names only; values are secrets)

Variable	Where the value comes from
NODE_ENV=production, PORT=8787, TRUST_PROXY=1, LOG_LEVEL=info	Fixed values from the template
DATABASE_URL, MIGRATE_DATABASE_URL	You build them from the Postgres passwords you generate. Host is the Compose service name; the app role and owner role differ
JWT_SECRET, METRICS_TOKEN, IP_HASH_SALT	You generate random values on the Mac
CORS_ORIGINS, APP_BASE_URL	https://app.<domain> (the Vercel address), from Nishit once the domain is live

Variable	Where the value comes from
Cookie domain (new setting, added in sub-phase 3c)	The parent domain, for example .<domain>
PAYPAL_ENV=sandbox, PAYPAL_API_BASE, PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET	Nishit, through a password manager. Sandbox credentials only
GEMINI_API_KEY, GROQ_API_KEY, OPENROUTER_API_KEY	Free keys created by team members, shared through a password manager
SIGNUPS_ENABLED, AI_ENABLED, AI_DAILY_CALLS	Kill switches and caps, from the template

6. Secrets, the one rule

Never put a password, key, token or tunnel token in git, in chat, in a screenshot, in an issue, or in this PDF. Secrets live in three places only: the Mac's `~/foldback/.env` (mode 600), a password manager, and the Vercel and Cloudflare dashboards. Share instructions for getting a secret, never the secret. If one leaks, tell Nishit immediately so it is rotated.

- Generate secrets on the Mac itself. Type or paste shared ones directly into the `.env` file with an editor.
- The Cloudflare tunnel token is a secret. It is shown once in the dashboard; Nishit sends it through a password manager or a one-time share link.
- A secret scanner (GitGuardian) runs on every pull request. Keep made-up sample ids obviously fake, for example SAMPLE-AUTH-ALEX.

7. Keeping the host safe

- **No inbound ports.** Do not port-forward anything on the router. The tunnel is outbound-only.
- **Postgres is private.** It must bind to 127.0.0.1 (or stay on the Docker network) and never be published on all interfaces.
- **No GitHub self-hosted runner.** The repository is public. A self-hosted runner on a public repository can execute code from a pull request on your Mac. We deploy by pulling published images instead.
- Keep Docker images from `ghcr.io/AnujSaha0111` only, and the official `postgres:16`. Do not run random images or scripts a web page or tool suggests.
- Do not use the Mac for anything sensitive while it is a server. Keep a normal password on every account.
- Reliability risk to be honest about: a home machine is less reliable than a cloud server (power, Wi-Fi, a reboot). Nishit will also record the demo video before the deadline, because the video is the judging artifact if the Mac is offline during judging.

8. Instructions for the friend's Claude

If you are Claude, helping the Mac mini owner: read this whole document, then follow these rules.

- **Start with git.** Make sure the clone is on `main` and run `git pull`. Read `docs/spec/README.md`, `docs/spec/quality.md`, `docs/spec/deploy.md` (parts say Azure; the plan is now the Mac mini and Vercel as described here, and Nishit's side is rewriting it) and anything newer in `docs/handoff/`.
- **Your scope is the Mac mini**: setup, Docker, the tunnel client, backups, launchd jobs, and checking health. Do not change product code in `backend/` or the frontend. If you think something in the code or spec is wrong, say so to the user and stop. When the spec and the code disagree, stop and ask.
- **Go one step at a time.** Give the user one command, wait for them to paste the output, then give the next. Explain what each command does in one sentence. Never run a destructive command (delete, reset, force) without saying what it removes and getting a clear yes.
- **Never handle secrets in chat.** Do not ask the user to paste a password, key or token. Have them type it into the `.env` file or a password manager themselves. Never print the contents of `.env`.

- **No AI attribution anywhere**: not in commits, pull requests, files, comments or documents. The team pushes and merges its own work; do not push or merge for them. If you add anything to the repository (for example a runbook), do it on a branch named `ops-topic` and let the user open the pull request.
- **Approved dependencies only.** Do not install project packages that are not on the approved list in `docs/spec/quality.md`. Installing Homebrew, OrbStack or cloudflared on the Mac is fine and expected. Never pipe a remote script into a shell without reading it first.
- **Free services only.** No paid provider, no card-backed key. PayPal is sandbox only.
- **Report facts, not guesses.** If a command fails, show the user the error and diagnose it before trying something else. Say clearly when something is done and verified, and when it is not.

9. What to send back to Nishit

When	Send
After A1	Chip (uname -m), macOS version, RAM, free disk, FileVault state. The owner's name
After A3 and A4	Confirmation that sleep is off, autorestart is on, and which FileVault and auto-login choice you made
After A6 and A7	Output of `docker compose version` and the `ps` table showing healthy, and the aarch64 result
After A8	Test count and how long the tests took
After A9	Whether SSH key login works for Nishit, and whether Tailscale is set up
After A10	Backup disk ready, and what the off-site copy will be
Always	Any error text you cannot solve. Never passwords, tokens or keys

10. Quick reference

Need	Command (run from the repository or ~/foldback)
Update the code	git checkout main then git pull
Start the dev databases	docker compose -f deploy/compose.dev.yml up -d
See running containers	docker ps
See backend logs (after 3f)	docker compose logs -f --tail 100 backend
Restart the backend (after 3f)	docker compose restart backend
Check health	curl -s http://localhost:8787/health/ready (expect {"status":"ok"})
Free disk	df -h /